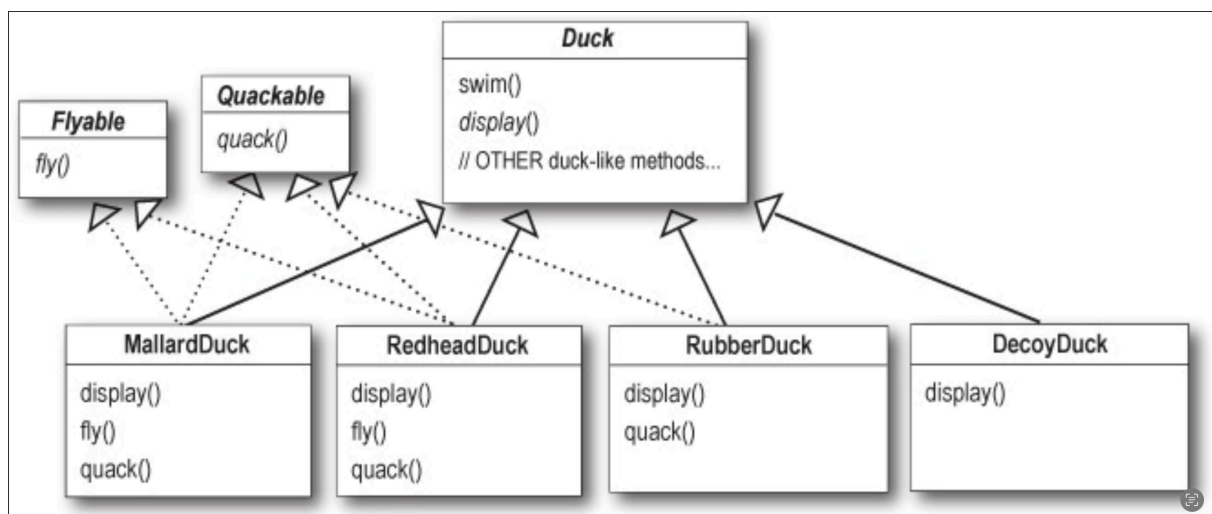


学習_Programming

デザインパターン strategy pattern

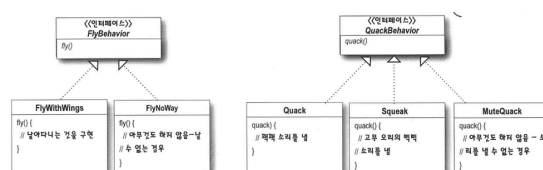
strategy patternは

変動する機能を別のオブジェクトに分離するデザインパターンである
継承だけを使う代わりに、合成を使用する
実行中にも挙動を変更することができる



最初は Duck クラスの中に `fly()` と `quack()` を直接実装した
しかし、**RubberDuck**は飛ばず、**DecoyDuck**も鳴くことができない
継承だけではすべてのケースに対応するのが難しくなる

そのため、行動をクラスに分離する



```

interface FlyBehavior{
    void fly();
}

class FlyWithWings implements FlyBehavior{
    public void fly(){
        System.out.println("날아갑니다.");
    }
}

class FlyNoWay implements FlyBehavior{
    public void fly(){
        System.out.println("날 수 없습니다.");
    }
}

class Duck{
    FlyBehavior flyBehavior;

    Duck(FlyBehavior flyBehavior){
        this.flyBehavior = flyBehavior;
    }

    void performFly(){
        flyBehavior.fly();
    }

    void setFlyBehavior(FlyBehavior flyBehavior){
        this.flyBehavior = flyBehavior;
    }
}

class Strategy{
    public static void main(String[] args) {
        Duck duck = new Duck(new FlyNoWay());

        duck.performFly();

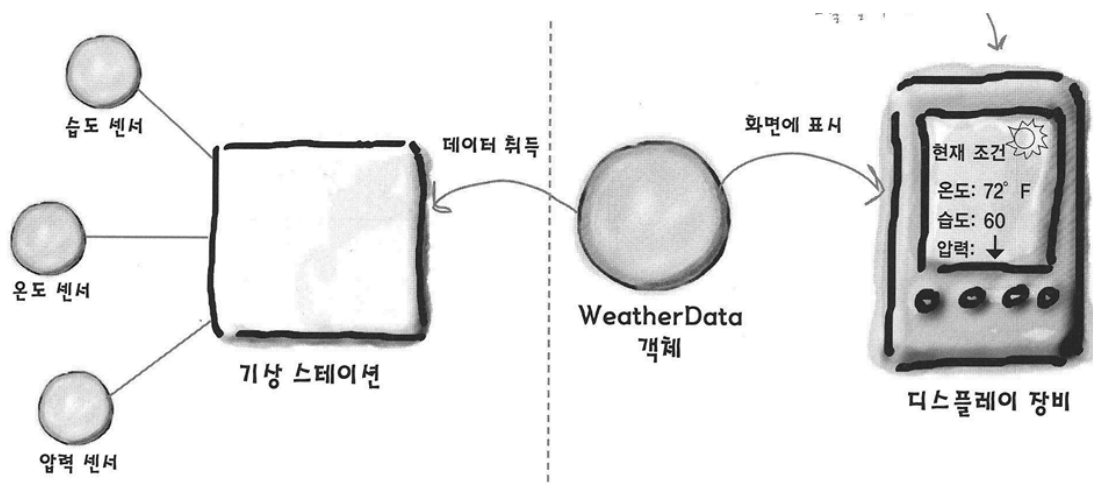
        duck.setFlyBehavior(new FlyWithWings());

        duck.performFly();
    }
}

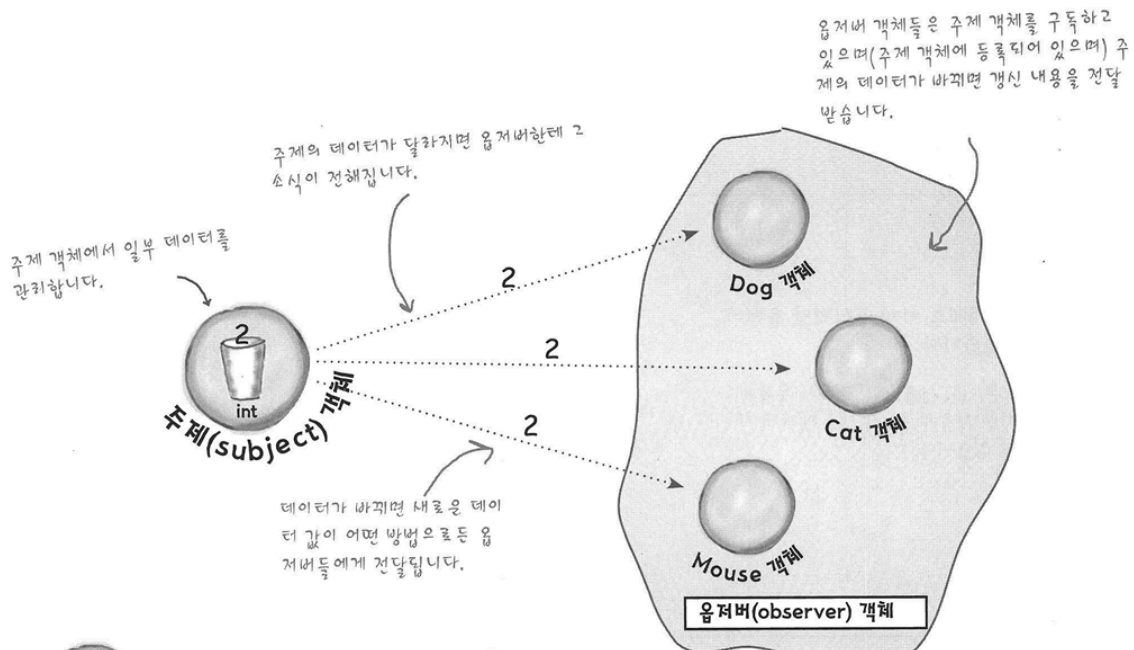
```

つくったら、このようになる

디자인 패턴 Observer Pattern



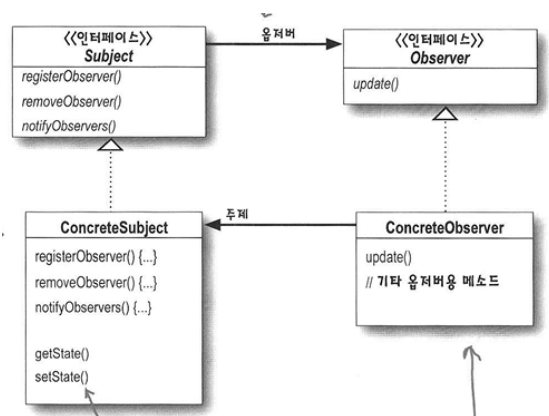
天気Stationを作るためにObserver Patternをつかう



Observer PatterにはSubjectとobserverがある

Subjectからデータを管理して、データの値が変わると、observerに知らせる

observerは一方的にこれを受け入れ



Observer Patternは大体このような仕組みになる

subjectインタフェースと、 observerインタフェースを使用

subjectインタフェースはobserverの登録と解消をする

observerインタフェースはsubjectの状態が変わる時のupdateメソッド実装

subjectクラスは登録、解消以外のメソッドとobserverに連絡するためのメソッドも実装する

observerは observer インタフェースの実装でobserverになる

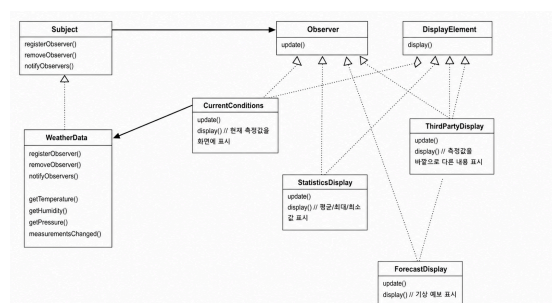
Observerクラスは LooseCoupling

Loose Couplingはお互いに相互作用はするけど互いに知らないことである

Observerをいつでも追加可能

Observerを追加する時、Subject変更は必要ない

お互いに独立的で再使用可能



Observer Patternを天気Stationに適用すればWeatherDataがSubject, DisplayはObserverになる

```
public interface Subject {

    public void registerObserver(Observer o);

    public void removeObserver(Observer o);

    public void notifyObservers();

}
```

```

import java.util.ArrayList;

public class WeatherData implements Subject {

    private ArrayList observers;
    private float temperature;
    private float humidity;
    private float pressure;

    public WeatherData() {
        observers = new ArrayList();
    }

    public void registerObserver(Observer o) {
        observers.add(o);
    }

    public void removeObserver(Observer o) {
        int i = observers.indexOf(o);
        if (i >= 0) {
            observers.remove(i);
        }
    }

    public void notifyObservers() {
        for (int i = 0; i < observers.size(); i++) {
            Observer observer = (Observer) observers.get(i);
            observer.update(temperature, humidity, pressure);
        }
    }

    public void measurementsChanged() {
        notifyObservers();
    }

    public void setMeasurements(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        this.pressure = pressure;
        measurementsChanged();
    }
}

```

このようにSubjectは登録と解消を作って他のメソッドを作る

```

public interface Observer {

    public void update(float temp, float humidity, float pressure);
}

```

```

public interface DisplayElement {

    public void display();
}

```

```

public class CurrentConditionDisplay implements Observer, DisplayElement{

    private float temperature;
    private float humidity;
    private Subject weatherData;

    public CurrentConditionDisplay(Subject weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }

    @Override
    public void display() {
        System.out.println("Current Conditions : " + temperature + "F degrees and " +
            humidity + "% humidity");
    }

    @Override
    public void update(float temp, float humidity, float pressure) {
        this.temperature = temp;
        this.humidity = humidity;
        display();
    }
}

```

```

public class StatisticsDisplay implements Observer, DisplayElement{

    private float temperature;
    private float humidity;
    private Subject weatherData;

    public StatisticsDisplay(Subject weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }

    @Override
    public void display() {
        System.out.println("Statistics : " + temperature + "F degrees and " +
            humidity + "% humidity");
    }

    @Override
    public void update(float temp, float humidity, float pressure) {
        this.temperature = temp;
        this.humidity = humidity;
        display();
    }
}

```

```

public class ForecastDisplay implements Observer, DisplayElement{

    private float temperature;
    private float humidity;
    private Subject weatherData;

    public ForecastDisplay(Subject weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }

    @Override
    public void display() {
        System.out.println("ForecastDisplay : " + temperature + "F degrees and " +
            humidity + "% humidity");
    }

    @Override
    public void update(float temp, float humidity, float pressure) {
        this.temperature = temp;
        this.humidity = humidity;
        display();
    }
}

```

このようにObserverもつくって

```

import java.util.Iterator;

public class WeatherStation {
    public static void main(String[] args) {
        WeatherData weatherData = new WeatherData();

        CurrentConditionDisplay currentDisplay =
            new CurrentConditionDisplay(weatherData);
        StatisticsDisplay statisticsDisplay = new StatisticsDisplay(weatherData);
        ForecastDisplay forecastDisplay = new ForecastDisplay(weatherData);

        weatherData.setMeasurements(80, 65, 30.4f);
        System.out.println("=====");
        weatherData.setMeasurements(82, 72, 29.2f);
        System.out.println("=====");
        weatherData.setMeasurements(78, 90, 29.2f);

    }
}

```

```
Current Conditions : 80.0F degrees and 65.0% humidity
Statistics : 80.0F degrees and 65.0% humidity
ForecastDisplay : 80.0F degrees and 65.0% humidity
=====
Current Conditions : 82.0F degrees and 72.0% humidity
Statistics : 82.0F degrees and 72.0% humidity
ForecastDisplay : 82.0F degrees and 72.0% humidity
=====
Current Conditions : 78.0F degrees and 90.0% humidity
Statistics : 78.0F degrees and 90.0% humidity
ForecastDisplay : 78.0F degrees and 90.0% humidity
```

SubjectがObserverに変わったデータを知らせるとこのようにデータを変える